

FastAPI: A Complete Beginner's Guide

1. What is FastAPI?

FastAPI is a modern, high-performance web framework for building **APIs (Application Programming Interfaces)** using Python.

Think of an API as a waiter in a restaurant. You (the client/frontend) look at the menu and tell the waiter what you want. The waiter takes your request to the kitchen (the backend/database) and brings your food back to you. FastAPI is the tool that helps you build that "waiter" quickly and efficiently.

It is built on two major pillars:

- **Starlette:** For the web handling parts (routing, handling requests, and sending responses).
- **Pydantic:** For validating the data to ensure it is exactly what your application expects.

(Insert Diagram: API Request/Response Cycle Workflow here)

2. What Does FastAPI Actually Do?

When you run a FastAPI application, it performs several critical jobs automatically behind the scenes:

1. **Handles Requests:** Listens for incoming web requests from browsers, mobile apps, or other servers.
2. **Validates Data:** Checks if the data sent to your API is correct. If your API expects a number for an age field but receives the text "young", FastAPI automatically blocks it and sends back a clear error message.
3. **Converts Data (Serialization):** Translates Python objects into JSON format (which the web understands) and vice versa.

4. **Generates Automatic Documentation:** The moment you write code, FastAPI automatically builds an interactive web page where you can see and test your API live.

3. Core Concepts to Understand

A. Routes and HTTP Methods

An API has different endpoints (URLs) that perform different actions using standard HTTP methods.

Method	Action	Example Use Case
GET	Retrieve data	Fetching a list of books
POST	Create new data	Adding a new book to the database
PUT / PATCH	Update existing data	Editing a book's price or description
DELETE	Remove data	Removing a book from the inventory

B. Path and Query Parameters

- **Path Parameter:** Used to identify a specific resource. (Example: `/books/42` where 42 is the exact book ID).
- **Query Parameter:** Used to filter, sort, or modify the data returned. (Example: `/books?author=Tolkien` where we filter books by author).

C. Pydantic Models

You use Python classes to define the exact "shape" of the data. Pydantic enforces these rules so you don't have to write manual if/else statements to check if the user sent the right data type.

D. Asynchronous Code (async and await)

FastAPI supports asynchronous programming out of the box. This allows your API to handle thousands of requests simultaneously. Instead of waiting for a slow database query to finish before serving the next user, FastAPI can pause that task, serve the next user, and come back when the database is ready.

4. Practical Task: Build a "Book Inventory" API

Follow these steps to build your first working API.

Step 1: Install Dependencies

Open your terminal or command prompt and install FastAPI and Uvicorn (the server that runs your code):

```
pip install fastapi uvicorn
```

None

Step 2: Write the Code

Create a new file named `main.py` and paste the following code:

```
```python
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI()

class Book(BaseModel):
 id: int
 title: str
 author: str
 is_available: bool = True

Mock Database
books_db = [
```

```

 {"id": 1, "title": "The Hobbit", "author": "J.R.R. Tolkien", "is_available":
True}
]

@app.get("/books", response_model=List[Book])
async def get_books():
 return books_db

@app.post("/books")
async def add_book(book: Book):
 books_db.append(book.dict())
 return {"message": "Book added successfully", "book": book}

```

## Step 3: Run Your API

Start your server by typing this into your terminal:

```

None
uvicorn main:app --reload

```

(Note: `--reload` makes the server restart automatically whenever you save changes to your file.)

## Step 4: Test it Live!

1. Open your web browser and go to: <http://127.0.0.1:8000/docs>
2. You will see the interactive Swagger UI.
3. Click on the **GET /books** endpoint, hit "Try it out", and click "Execute".
4. Try using the **POST /books** endpoint to add your own custom book!

## Some key definition to know:-

1. **asynchronous programming:-** FastAPI natively supports asynchronous programming using Python's `async` and `await` keywords, allowing a single server instance to handle thousands of concurrent requests without blocking.
2. **Serialization:-** FastAPI handles data serialization entirely through [Pydantic](#), a powerful library that converts complex Python objects (like database models, classes, and datetimes) into JSON-compatible formats.

